Machine Learning Techniques for Distracted Driver Detection

ML Mini Project · Team F-28 · Project 75 · Section F
Chiranth R (PES1UG24AM354) · Vihaan Wadhwa (PES1UG24AM319)

Abstract

We classify dashboard-camera photos of drivers into ten activities (safe driving and nine kinds of distraction) using the State Farm Distracted Driver Detection dataset (22,424 labelled images of 26 drivers). Following a Stanford CS229 project, we train classical models on flattened 64×64 pixels (Gaussian Naive Bayes, linear SVM, softmax regression, decision tree) and fine-tune an ImageNet-pretrained MobileNetV3-Large. Our main contribution is the evaluation: every model is trained and tested on two 70/15/15 splits of the same data, a random split in which the same drivers appear in training and test, and a driver-grouped split in which test drivers are never seen. On the random split the best model (MobileNetV3) reaches 99.9% test accuracy. On unseen drivers MobileNetV3 reaches 87.5%, 12.4 percentage points lower than on its random split (99.9%). Random splits therefore overstate how well such models work on new people. Grad-CAM visualisations show where each model looks. Driver-grouped 5-fold cross-validation with 3 seeds confirms the gap (85.5% ± 2.8 on unseen drivers). An adversarial driver classifier reduces identity information in the features but improves unseen-driver accuracy by only +0.8 points, which is not statistically significant.

1. Introduction

Distracted driving, such as texting, phone calls, drinking or reaching behind, takes the driver's attention away from the road. A camera-based system that recognises these activities could warn the driver in time. We study this as image classification: given one photo taken from the passenger side, predict which of ten activities the driver is doing.

Many reported results on this dataset use a random split of the photos. Because each driver contributes hundreds of near-identical frames, a random split places photos of the same person in both the training and the test set. Our question is how much of the measured accuracy survives when the test drivers are new. We answer it by training every model twice, on a random split and on a driver-grouped split, and comparing.

2. Related Work

Our reference is a Stanford CS229 (Spring 2019) project on the same dataset [1], which flattened 64×64×3 images and compared Naive Bayes, a linear SVM, softmax regression, a decision tree and a two-layer neural network, reporting a best accuracy of about 92% on a random 80/20 split. Convolutional networks pretrained on ImageNet [5] are the standard approach for image classification; we use MobileNetV3 [3], a small network designed for mobile devices, which suits an in-car setting. To inspect what the networks rely on we use Grad-CAM [4]. Keeping all images of one subject in the same split (a subject-wise or grouped split) is a standard precaution against identity leakage when a dataset contains many images of few people.

3. Dataset and Preprocessing

We use the labelled training set of the Kaggle competition "State Farm Distracted Driver Detection" [2]: 22,424 RGB photos (640×480) of 26 drivers in ten classes. The file driver_imgs_list.csv gives the driver ID (subject) of each photo. Classes are roughly balanced, from 1,911 to 2,489 photos per class (Figure 1). The competition's unlabelled test images are not used.

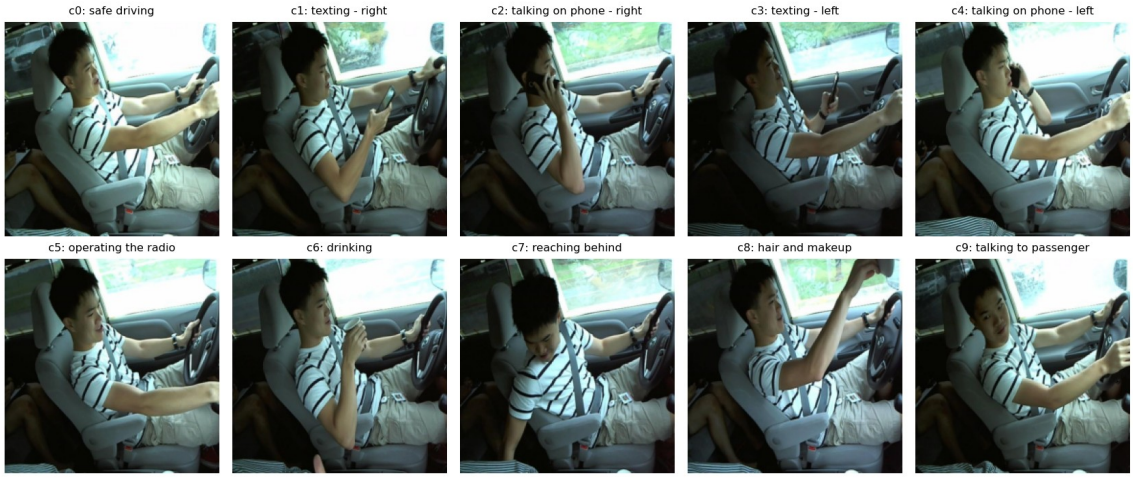


Figure 1: One example photo per class (c0 safe driving … c9 talking to passenger).

Splits: Both splits use 70% / 15% / 15% for train / validation / test with random seed 42. The random split is stratified by class and ignores the driver. The driver split assigns whole drivers: 18 drivers to training, 4 to validation and 4 to test (test drivers: p022, p039, p047, p056). Table 1 gives the sizes.

Table 1: Sizes of the two splits. In the random split every driver appears in every part.

Split	Part	Photos	Share	Drivers
random	train	15,696	70.0%	26
random	val	3,364	15.0%	26
random	test	3,364	15.0%	26
driver	train	15,313	68.3%	18
driver	val	3,598	16.1%	4
driver	test	3,513	15.7%	4

Preprocessing: For the classical models each photo is resized to 64×64×3, flattened into 12,288 values in [0, 1] and standardised with a scaler fitted on the training set only. For MobileNetV3 photos are resized to 224×224 and normalised with the ImageNet mean and standard deviation.

4. Methods

Classical baselines

Following [1] we train four models with scikit-learn [6]: Gaussian Naive Bayes; a linear SVM (hinge loss, stochastic gradient descent, L2 penalty 1e-4); softmax regression (multinomial logistic regression, L-BFGS); and a decision tree (unlimited depth). For speed each model trains on a class-

stratified subsample of the training set: 8,000 photos for the random split and 8,000 for the driver split. No hyperparameters are tuned.

MobileNetV3-Large

We fine-tune all layers of an ImageNet-pretrained MobileNetV3-Large [3] in PyTorch [7], replacing the last layer with a 10-way output. Training uses AdamW [8] with weight decay 0.05, a one-cycle learning-rate schedule peaking at 0.001, batch size 64, label smoothing 0.1, mixed precision, and 8 epochs on a Tesla T4 GPU. Augmentation is light: random affine shifts, rotations (up to 8°) and scaling, plus colour jitter. We do not flip images horizontally, because that would turn 'texting with the right hand' into 'texting with the left hand'. The checkpoint with the best validation accuracy is kept (epoch 7 for the random split, epoch 7 for the driver split). The test set is used only once, for the final numbers.

Evaluation

For every model and split we report test accuracy, macro-averaged F1, per-class accuracy and the confusion matrix. For the driver split we also report accuracy for each test driver. Grad-CAM [4] heatmaps are computed on the last convolutional block for twelve photos that neither model saw during training (they are in the random-split test set and belong to a driver-split test driver).

5. Experiments and Results

Table 2: Test results. Random-split test set: 3,364 photos; driver-split test set: 3,513 photos of 4 unseen drivers.

Model	Random acc	Driver acc	Drop (pts)	F1 random	F1 driver
Naive Bayes	51.1%	26.0%	25.1	0.508	0.228
Linear SVM	95.9%	33.0%	62.8	0.959	0.338
Softmax	97.9%	34.3%	63.6	0.979	0.334
Decision Tree	74.6%	30.4%	44.2	0.748	0.303
MobileNetV3	99.9%	87.5%	12.4	0.999	0.872

Table 2 and Figure 2 summarise the results. Naive Bayes drops from 51.1% to 26.0%; Linear SVM drops from 95.9% to 33.0%; Softmax drops from 97.9% to 34.3%; Decision Tree drops from 74.6% to 30.4%; MobileNetV3 drops from 99.9% to 87.5%.

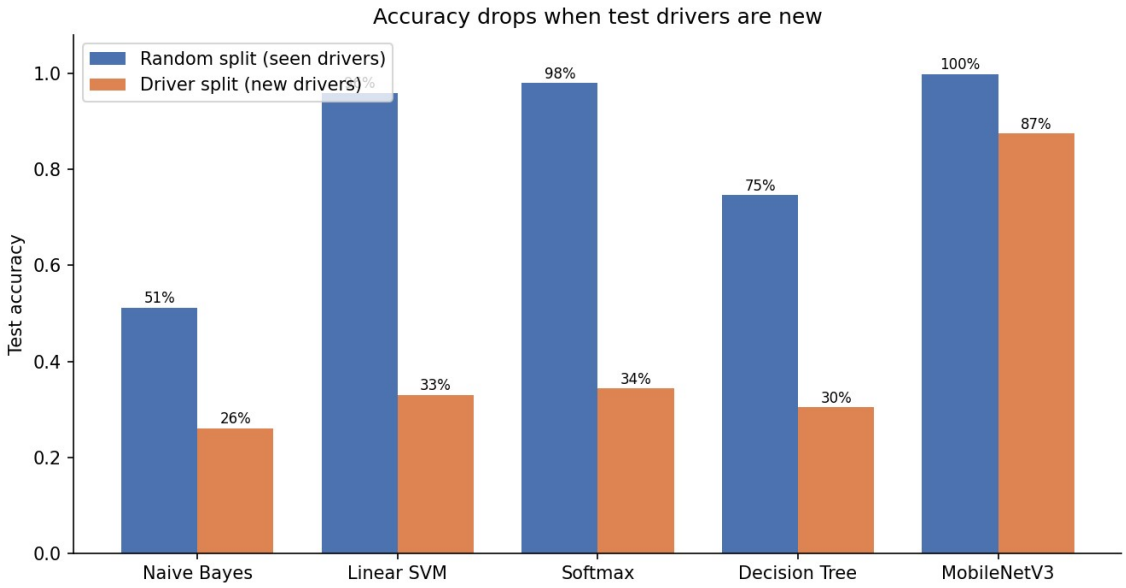


Figure 2: Test accuracy of every model on the random split (seen drivers) and the driver split (new drivers).

Table 3: MobileNetV3 per-class test accuracy on both splits.

Class	Name	Random split	Driver split
c0	safe driving	100.0%	93.8%
c1	texting - right	100.0%	99.4%
c2	talking on phone - right	100.0%	81.1%
c3	texting - left	99.7%	98.3%
c4	talking on phone - left	99.7%	99.4%
c5	operating the radio	100.0%	98.3%
c6	drinking	100.0%	82.8%
c7	reaching behind	100.0%	99.4%
c8	hair and makeup	100.0%	71.2%
c9	talking to passenger	99.4%	50.6%

On new drivers the hardest classes for MobileNetV3 are c9 (talking to passenger, 50.6%), c8 (hair and makeup, 71.2%), c2 (talking on phone - right, 81.1%). Figure 3 shows the confusion matrix.

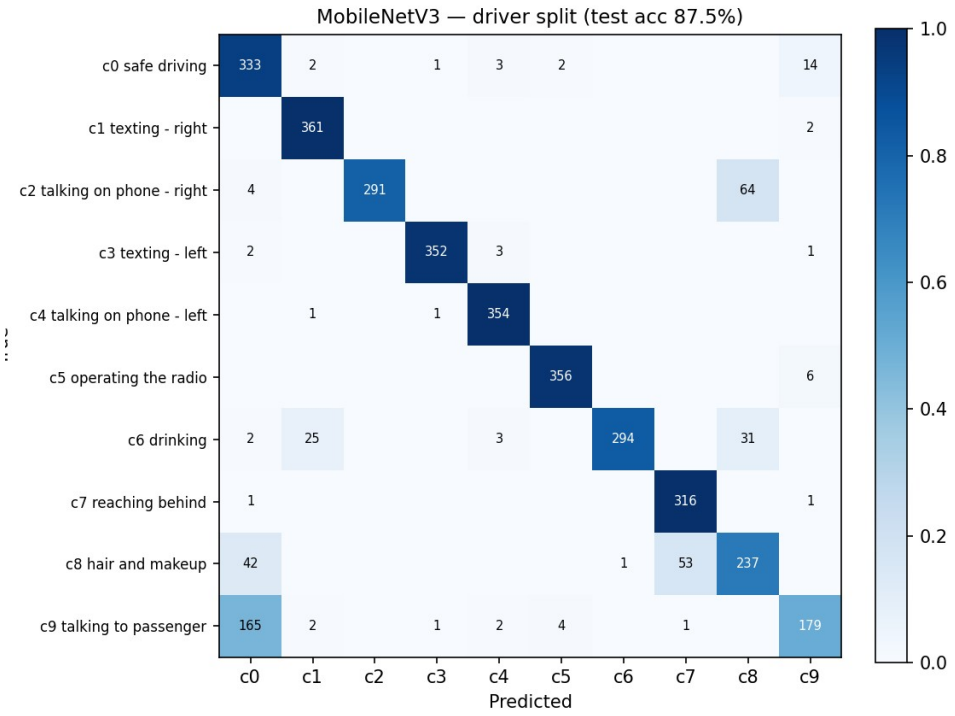


Figure 3: Confusion matrix of MobileNetV3 on the driver split (rows: true class, columns: predicted).

Accuracy also varies between test drivers: p022 92.4% (1233 photos), p039 79.4% (651 photos), p047 90.5% (835 photos), p056 83.2% (794 photos) (Figure 4).

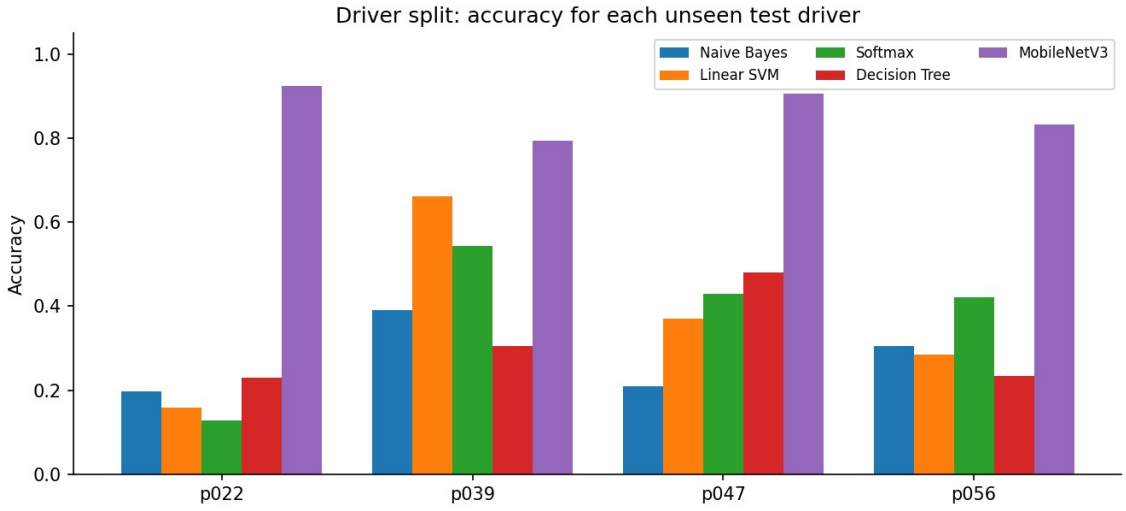


Figure 4: Accuracy of each model on each unseen test driver (driver split).

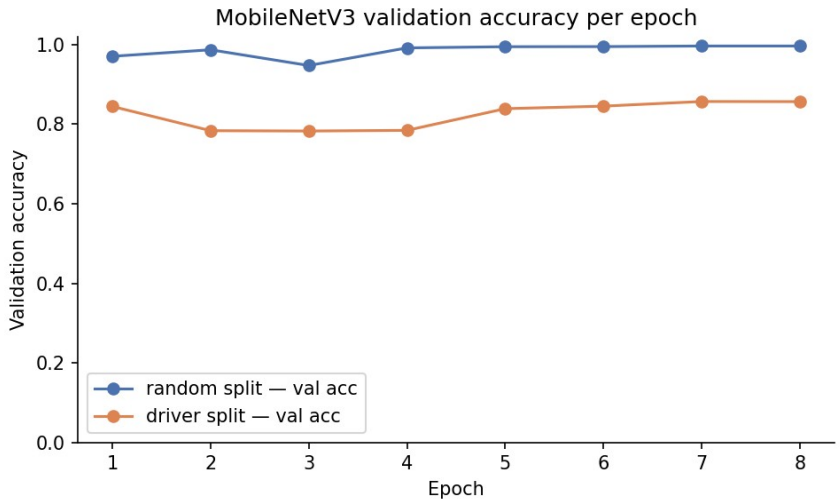


Figure 5: MobileNetV3 validation accuracy per epoch on both splits.

Figure 6 shows Grad-CAM heatmaps for 12 photos unseen by both models. On these photos the random-split model is correct on 12 and the driver-split model on 10.

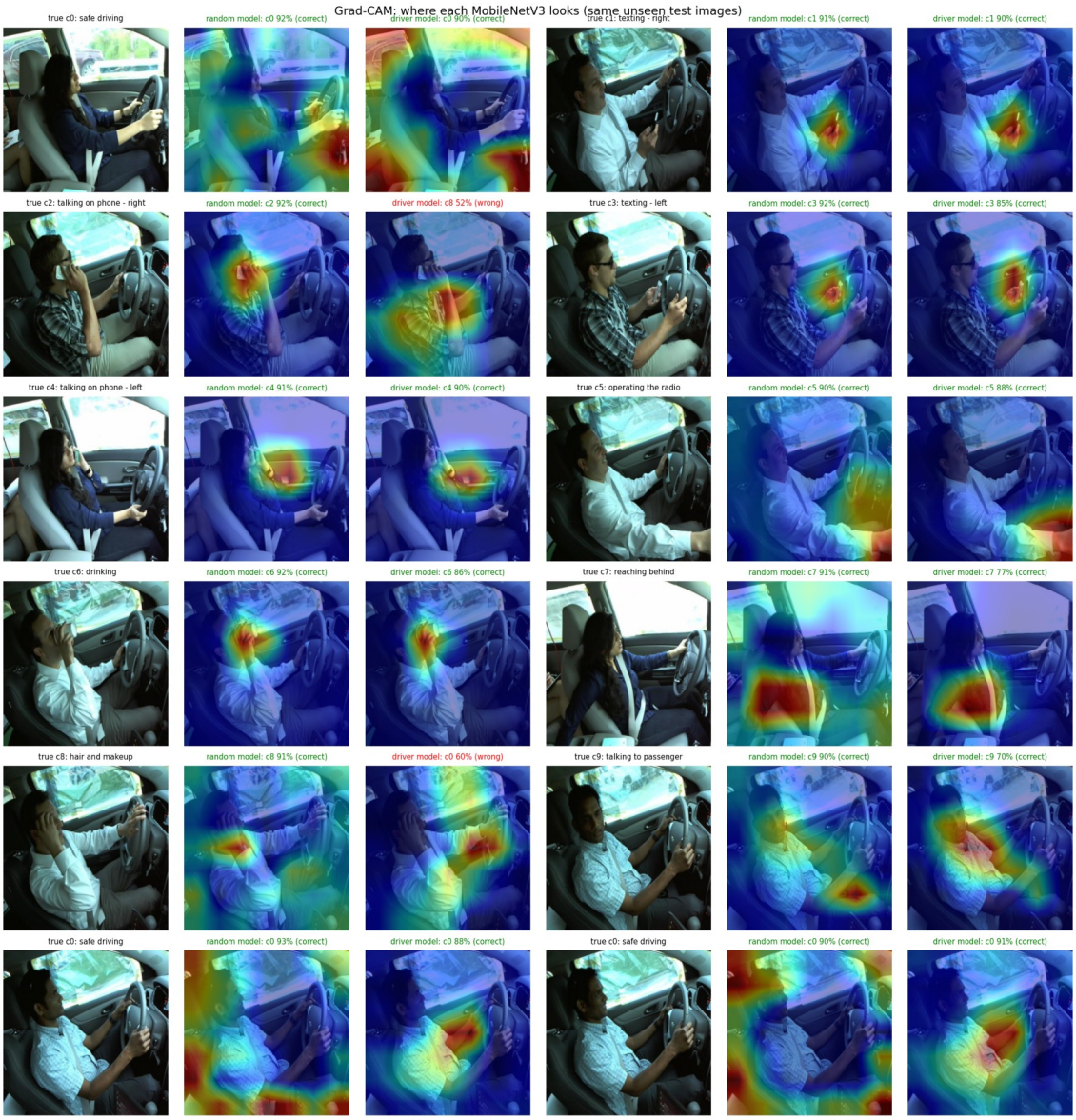


Figure 6: Grad-CAM for the random-split model and the driver-split model on the same unseen photos.

Can the model learn the action without learning the people?

We tested two ways to reduce reliance on driver appearance, training MobileNetV3 on the same driver split with the same optimiser and 8 epochs. (A) Appearance-scrambling augmentation: wider brightness/contrast/saturation jitter, random grayscale ($p = 0.2$) and random erasing of a 2–20% patch ($p = 0.5$), still with no horizontal flip. (B, C) The same augmentation plus an adversarial driver classifier: a small head predicts which of the 18 training drivers is in the photo from the 960-d pooled features, behind a gradient-reversal layer, so the backbone is trained to make those features useless for identifying the driver (domain-adversarial training [10]); the reversal strength ramps up to $\lambda = 0.3$ (B) or $\lambda = 1.0$ (C). Each variant keeps its best epoch by validation accuracy and the final variant is chosen by validation accuracy only; Table 4 reports all of them.

Table 4: Debiasing variants on the driver split (unseen drivers). The last four columns give accuracy per test driver.

Variant	Validation acc	Test acc	Test macro-F1	p022	p039	p047	p056
Baseline (main run)	85.7%	87.5%	0.872	92.4%	79.4%	90.5%	83.2%
A: augmentation	86.8%	90.3%	0.898	90.1%	83.1%	94.4%	92.3%
B: A + adversary ($\lambda \leq 0.3$)	89.4%	89.2%	0.889	90.2%	89.6%	89.5%	87.0%
C: A + adversary ($\lambda \leq 1.0$) (chosen)	90.1%	91.5%	0.914	95.7%	86.6%	86.7%	93.8%

The chosen variant (C: A + adversary ($\lambda \leq 1.0$)) reaches 91.5% test accuracy on unseen drivers, compared with 87.5% for the baseline. The largest per-class gains are c9 (talking to passenger) 50.6% → 68.6%, c8 (hair and makeup) 71.2% → 88.9%, c6 (drinking) 82.8% → 94.9%; the largest loss is c0 (safe driving) 93.8% → 73.8%, which in a real system would mean more false alarms for safe drivers. During training the adversarial head's driver accuracy fell to near chance (1/18), but a separate identity probe (logistic regression on frozen features, telling the 4 test drivers apart) scored 99.9%–100.0% for every model including the ImageNet-only network, so this probe is saturated and cannot show whether identity information was reduced. With 4 test drivers and one seed, the improvement is promising but not yet statistically established; the next subsection tests it with cross-validation.

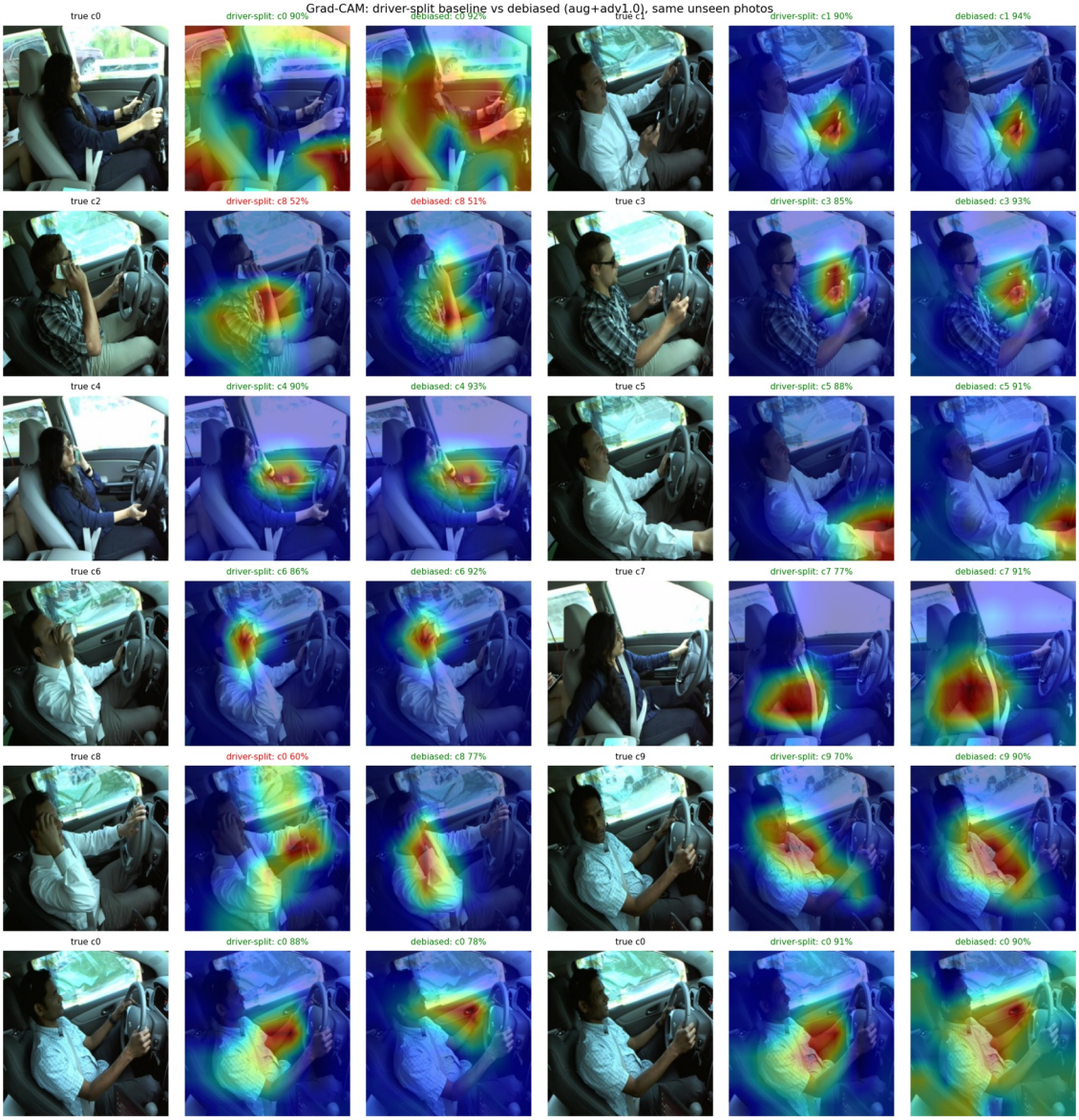


Figure 7: Grad-CAM of the driver-split baseline and the debiased model on the same unseen photos.

Is the improvement real? Driver-grouped cross-validation

A single split with 4 test drivers is noisy, so we repeated the comparison with 5-fold driver-grouped cross-validation. The 26 drivers were shuffled once (seed 42) and cut into 5 groups (6, 5, 5, 5, 5 drivers). In fold k, group k is the unseen test set, group k+1 is the validation set used to pick the epoch, and the other three groups are used for training. Every fold was run with 3 seeds (45 training runs). Differences against the baseline are tested at fold level (seeds averaged, so the 5 folds are the independent units) with a paired t-test; with only 5 folds these tests have little power. We also measured identity information on each fold's unseen drivers with two harder probes on the frozen, L2-normalised 960-d features. One-shot identification takes one reference photo per test driver within the same action class, and assigns every other photo of that class to the nearest reference by cosine similarity; chance is about 1 in 5. Same-person verification measures the ROC AUC of cosine

similarity for same-driver versus different-driver pairs within a class (0.5 means identity cannot be told apart). Both probes are for analysis only and never used for model selection.

Table 5: Driver-grouped 5-fold cross-validation, 3 seeds per fold. Accuracy is on unseen drivers. The last two columns are identity probes on unseen drivers (one-shot chance ≈ 0.19; verification chance = 0.5); lower means less identity information.

Model	Acc % (±sd)	F1	Δ pts [95% CI]	p	1-shot ID	Verif. AUC
Baseline	85.5 ± 2.8	0.844	–	–	0.856	0.873
A: scramble	85.7 ± 3.4	0.851	+0.2 [-2.4, +2.8]	0.86	0.865	0.885
C: A + adversary	86.3 ± 3.1	0.857	+0.8 [-0.9, +2.5]	0.26	0.815	0.859
ImageNet only	–	–	–	–	0.992	0.993

Averaged over all 26 drivers, the baseline reaches 85.5% on unseen drivers, compared with 99.9% on the random split, so the leakage gap is robust to the choice of test drivers. The debiasing gains are much smaller than on the single split. Model C is +0.8 points better on average, is better on 4 of 5 folds and in 10 of 15 runs, but the difference is not significant (p = 0.26). Model A is essentially unchanged (+0.2 points, p = 0.86). The single-split gain therefore came largely from which 4 drivers happened to be in the test set.

The identity probes show that the ImageNet features identify drivers almost perfectly (99.2% one-shot). Fine-tuning on the actions reduces this to 85.6%. The adversary reduces it further, to 81.5%, lower than the baseline on all 5 folds (paired t-test p = 0.006). The verification AUC also falls slightly (-0.014, p = 0.13). The adversary therefore does remove some identity information, but identity remains far above chance, and removing part of it buys only a small, uncertain gain in accuracy. Safe-driving accuracy (c0) averages 82.3% for the baseline, 75.4% for A and 80.7% for C.

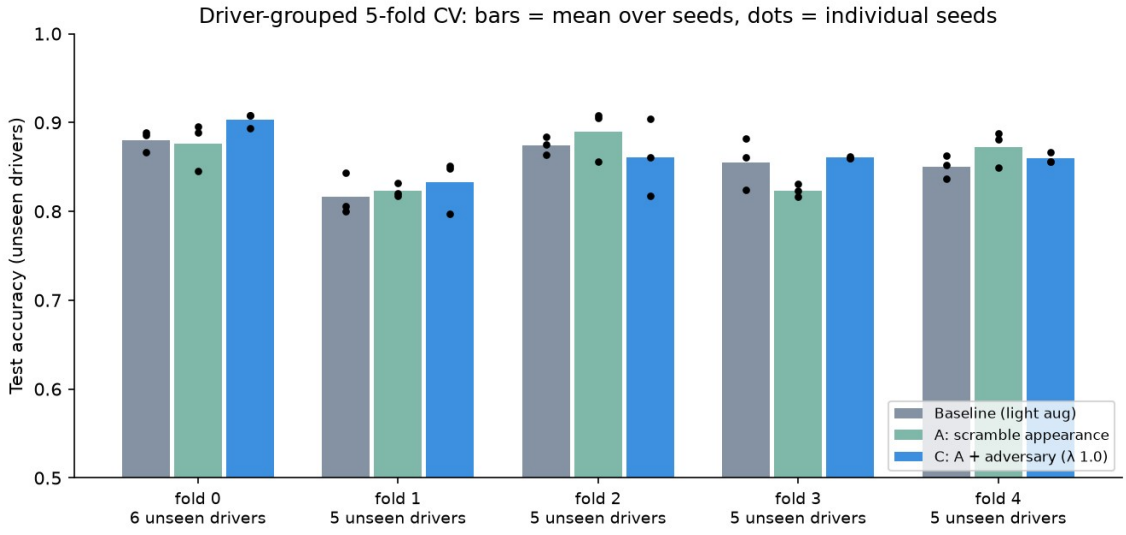


Figure 8: Cross-validation: unseen-driver accuracy per fold (bars: mean over seeds; dots: individual seeds).

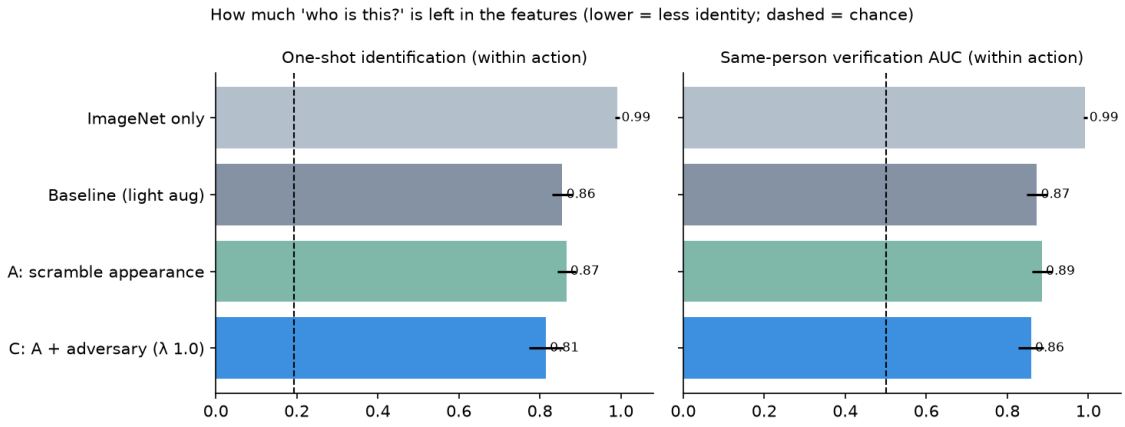


Figure 9: Identity probes on unseen drivers (lower = less identity information; dashed lines = chance).

Compute: the complete Kaggle run took 14.4 minutes on one Tesla T4.

6. Discussion: why random splits inflate accuracy

In this dataset each driver is filmed in long sequences, so neighbouring photos are almost identical apart from the hand or head position. With a random split, most test photos have near-duplicates of the same person, in the same clothes, seat position and lighting, in the training set. A model can then score well by matching appearance and background, without fully learning the activity. The driver split removes this shortcut: the person, clothes and posture are new, and only the activity carries over. The gap between the two columns of Table 2 measures how much of the random-split accuracy came from recognising seen people. For MobileNetV3 the gap is 12.4 percentage points; for the classical pixel-based models, which can only compare raw pixels, see the Drop column of Table 2.

A deployed system will always meet new drivers, so the driver-split number is the honest estimate of real-world accuracy. Results on this dataset should state which split was used.

7. Limitations

- Only 26 drivers, with 4 in the driver-split test set, so the driver-split accuracy depends on which drivers were held out.
- Cross-validation used 5 folds and 3 seeds, so the significance tests have little power; small effects such as the debiasing gain need more folds or more drivers to confirm.
- Only one dataset was used. Testing on a second dataset (e.g. the AUC Distracted Driver Dataset, available under licence) is needed to check that the findings generalise.
- Classical models were trained on a subsample of 8,000 photos and were not tuned.
- All photos come from one camera setup; a different car, camera angle or lighting would be a further distribution shift.
- Grad-CAM shows which regions raised the predicted score; it is a rough explanation, not a proof of the model's reasoning.

8. Conclusion and Future Work

We reproduced the classical baselines of [1], added a pretrained MobileNetV3, and evaluated every model on a random split and on a driver-grouped split. Random-split accuracy (best: MobileNetV3, 99.9%) overstates performance on new people; on unseen drivers MobileNetV3 reaches 87.5%. Distracted-driver models should be evaluated with driver-grouped splits.

Five-fold driver-grouped cross-validation confirms that the leakage gap is robust (85.5% ± 2.8 on unseen drivers). An adversarial driver classifier measurably reduces identity information in the features, but its accuracy gain (+0.8 points) is not significant. A promising single-split result (+4 points) shrank under cross-validation, which is itself a warning about evaluating on a few drivers.

Future work: more drivers and a second dataset (cross-dataset testing); stronger ways to separate the action from the person, such as pose or hand-region inputs; more folds and seeds for statistical power. As our next research step, we plan to test vision-language models such as CLIP [9] with driver-appearance decoupling under realistic calibration, to check whether language-aligned features recognise the activity rather than the person.

9. Contributions of Each Member

Chiranth R (PES1UG24AM354) and Vihaan Wadhwa (PES1UG24AM319) contributed equally to this project. Both members shared the work on data preparation and the two evaluation splits, training and evaluating the classical baselines and MobileNetV3, the debiasing and cross-validation experiments, the live demo application, and the slides and this report.

There is clear scope for future research on this work, and both members intend to continue it: testing the findings on a second dataset with new drivers (the AUC Distracted Driver Dataset, requested under licence), trying stronger ways to remove driver identity from the model (such as concept erasure and pose- or hand-based inputs), and evaluating vision-language models such as CLIP with driver-appearance decoupling under realistic calibration.

References

[1] Stanford CS229 Machine Learning, Spring 2019, Project Report 24: distracted driver detection. https://cs229.stanford.edu/proj2019spr/report/24.pdf (poster: https://cs229.stanford.edu/proj2019spr/poster/24.pdf)

[2] State Farm Distracted Driver Detection, Kaggle competition, 2016. https://www.kaggle.com/c/state-farm-distracted-driver-detection

[3] A. Howard et al., "Searching for MobileNetV3," in Proc. IEEE/CVF International Conference on Computer Vision (ICCV), 2019.

[4] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," in Proc. ICCV, 2017.

[5] J. Deng et al., "ImageNet: A Large-Scale Hierarchical Image Database," in Proc. IEEE CVPR, 2009.

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, 2011.

[7] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in NeurIPS, 2019.

[8] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in Proc. ICLR, 2019.

[9] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," in Proc. ICML, 2021.

[10] Y. Ganin and V. Lempitsky, "Unsupervised Domain Adaptation by Backpropagation," in Proc. ICML, 2015.